

BUILDING A SIMPLE RAG SYSTEM WITH PYTHON

Student Instruction Guide

PART 1: Document -> Chunk -> Embed -> Pinecone
PART 2: Question -> Embed -> Search -> Context -> Gemini -> Answer

Uses Gemini Embedding 2 + Pinecone + Python

1. Lab Overview

In the previous Make.com activity, you built two visual workflows for Retrieval-Augmented Generation (RAG). In this lab, you will implement the same ideas manually in Python so you can see what each Make module was doing behind the scenes.

Core idea: Do not treat this as a new architecture. The goal is to translate the two Make.com pipelines into code, one operation at a time.

RAG has two main stages:

- Part 1 - Ingestion: prepare a document so it can be searched semantically.
- Part 2 - Query: retrieve relevant document chunks and give them to a generative model as context.

PART 1 - INGESTION

Text file -> Chunking -> Gemini embedding -> Pinecone records

PART 2 - QUERY

Question -> Gemini embedding -> Pinecone similarity search -> Top-k chunks -> Gemini answer

Learning outcomes

By the end of this lab, you should be able to:

- explain why a RAG system needs a separate ingestion pipeline and query pipeline;
- implement simple overlapping text chunking in Python;
- convert document chunks and questions into embedding vectors;
- store vectors and metadata in a Pinecone index and namespace;
- perform similarity search with top-k retrieval;
- aggregate retrieved chunks into a context prompt;
- generate an answer that is grounded in retrieved context;
- map each Python operation back to the Make.com module you used previously.

2. Make.com to Python: the Direct Mapping

Make.com - Part 1	Python equivalent	Purpose
Google Drive - Download a File	Path(...).read_text()	Load document content
AI Toolkit - Chunk text	chunk_text()	Split document into smaller overlapping pieces
Iterator	for chunk in chunks	Process one chunk at a time
Gemini - Generate embeddings	embed_text(chunk)	Convert text into a numeric vector
Pinecone - Upsert vector	index.upsert(...)	Store vector and metadata

Make.com - Part 2	Python equivalent	Purpose
Tools - Set variable	<code>input(...)</code>	Capture a test question
Gemini - Generate embeddings	<code>embed_text(question)</code>	Represent the question as a vector
Pinecone - Search with a query	<code>index.query(...)</code>	Find similar document chunks
Tools - Text aggregator	<code>'\n\n--\n\n'.join(...)</code>	Combine retrieved chunks into context
Gemini - Generate a response	<code>gemini.interactions.create(...)</code>	Answer using the retrieved context
Tools - Get variable	<code>print(answer)</code>	Display the final answer

Important: The code is intentionally manual. Do not use LangChain or LlamaIndex in this lab. Those frameworks are useful later, but they would hide the exact operations you are learning here.

3. Before You Start

3.1 Accounts and materials

- A Gemini API key (the same service used in the Make.com lab).
- A Pinecone account and API key.
- Python 3 installed on your computer.
- The provided `intelligent_agents_overview.txt` file from the Week 3 lab materials.
- A code editor such as Visual Studio Code.

3.2 Why we read the file locally

The Make.com ingestion scenario starts with Google Drive. In Python, we will read the same `.txt` file from a local data folder. This keeps the lab focused on RAG rather than Google OAuth and Drive API authentication.

Conceptual equivalence: Make.com Google Drive -> Download a File and Python `Path.read_text()` serve the same role in this lab: both produce the document text that is sent to the chunking stage.

4. Set Up the Python Project

4.1 Create the project folders

```
simple-rag/
|-- data/
|   |-- intelligent_agents_overview.txt
|-- ingest.py
|-- query.py
|-- .env
|-- .gitignore
-- requirements.txt
```

4.2 Create a virtual environment

Windows PowerShell:

```
python -m venv .venv  
.venv\Scripts\Activate.ps1
```

macOS / Linux:

```
python3 -m venv .venv  
source .venv/bin/activate
```

4.3 Install the required packages

```
pip install -U google-genai pinecone python-dotenv
```

requirements.txt

```
google-genai  
pinecone  
python-dotenv
```

4.4 Store API keys safely

Create a file called `.env` in the project root:

```
GEMINI_API_KEY=your_gemini_api_key  
PINECONE_API_KEY=your_pinecone_api_key  
PINECONE_INDEX=your_index_name
```

Create `.gitignore`:

```
.env  
.venv/  
__pycache__/
```

Security: Never commit `.env` or paste real API keys into source code, screenshots, GitHub, or your submission.

5. Prepare the Pinecone Index

You may reuse the dense Pinecone index from the Make.com lab if it was created with dimension 1024. Otherwise, create a new dense index in the Pinecone console.

Setting	Value for this lab	Reason
Vector type	Dense	Gemini produces dense embedding vectors
Dimension	1024	Matches the Make.com blueprint and our Python embedding output
Metric	Cosine	Compares vector direction / semantic similarity
Namespace	week3-code-lab	Keeps Python records separate from the Make.com records

Why 1024?: Gemini Embedding 2 defaults to 3072 dimensions and Google currently recommends 768, 1536, or 3072. However, Gemini Embedding 2 supports flexible output dimensions from 128 to 3072. We intentionally use 1024 here because the supplied Make.com blueprint uses 1024, so both labs stay directly comparable.

Three values must stay consistent:

- Part 1 embedding dimension = 1024.
- Part 2 embedding dimension = 1024.
- Pinecone index dimension = 1024.

Also keep the embedding model consistent: Use gemini-embedding-2 for both document chunks and questions. Vectors from different embedding models may live in different semantic spaces even when they have the same numeric length.

Namespace reminder: a namespace is a logical partition inside an index. Pinecone creates the namespace automatically when you first upsert records into it.

6. Part 1 - Build the Ingestion Pipeline

File -> Read text -> Chunk -> Iterate -> Embed -> Upsert to Pinecone

6.1 Imports and configuration

```
import os
from pathlib import Path

from dotenv import load_dotenv
from google import genai
from google.genai import types
from pinecone import Pinecone

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
INDEX_NAME = os.getenv("PINECONE_INDEX")

NAMESPACE = "week3-code-lab"
EMBEDDING_MODEL = "gemini-embedding-2"
EMBEDDING_DIMENSION = 1024
```

What this replaces in Make: the account connections and module settings configured through the Make.com UI.

6.2 Validate the configuration and connect

```
if not all([GEMINI_API_KEY, PINECONE_API_KEY, INDEX_NAME]):
    raise ValueError(
        "Missing GEMINI_API_KEY, PINECONE_API_KEY, or PINECONE_INDEX in .env"
    )

gemini = genai.Client(api_key=GEMINI_API_KEY)
pc = Pinecone(api_key=PINECONE_API_KEY)
index = pc.Index(INDEX_NAME)
```

Lab simplification: Targeting a Pinecone index by name is convenient for teaching and testing. Production applications should normally resolve and reuse the index host rather than repeatedly targeting by name.

6.3 Write the chunking function

The Make blueprint used Chunk size = 200 and Chunk overlap = 20. The blueprint does not tell us the internal unit used by the Make module. In this Python lab, we define the unit explicitly as words so the algorithm is transparent.

```
def chunk_text(text: str, chunk_size: int = 200, overlap: int = 20) -> list[str]:
    """Split text into overlapping word-based chunks."""
    if chunk_size <= 0:
        raise ValueError("chunk_size must be greater than 0")
    if overlap < 0 or overlap >= chunk_size:
        raise ValueError("overlap must be >= 0 and smaller than chunk_size")

    words = text.split()
    chunks = []
    step = chunk_size - overlap

    for start in range(0, len(words), step):
        end = start + chunk_size
        chunk_words = words[start:end]

        if not chunk_words:
            break

        chunks.append(" ".join(chunk_words))

    if end >= len(words):
        break

    return chunks
```

Why overlap?: With `chunk_size=200` and `overlap=20`, the next chunk begins 180 words after the previous one. The final 20 words of one chunk are repeated at the start of the next, reducing context loss at chunk boundaries.

6.4 Create the embedding function

```
def embed_text(text: str) -> list[float]:
    result = gemini.models.embed_content(
        model=EMBEDDING_MODEL,
        contents=text,
        config=types.EmbedContentConfig(
            output_dimensionality=EMBEDDING_DIMENSION
        ),
    )
    return result.embeddings[0].values
```

Conceptually: the input is ordinary text; the output is a list of 1024 floating-point numbers that represents the semantic meaning of that text.

"An intelligent agent perceives..." -> Gemini Embedding 2 -> [0.021, -0.184, ..., 0.063]

6.5 Read the document

```
file_path = Path("data/intelligent_agents_overview.txt")

if not file_path.exists():
    raise FileNotFoundError(
        f"Cannot find {file_path}. Put the lab text file inside the data folder."
    )

text = file_path.read_text(encoding="utf-8")
print(f"Loaded: {file_path.name}")
print(f"Characters: {len(text)}")
```

Make mapping: this replaces Google Drive -> Download a File.

6.6 Chunk the document

```
chunks = chunk_text(text, chunk_size=200, overlap=20)
print(f"Created {len(chunks)} chunks")
```

Optional inspection while learning:

```
for i, chunk in enumerate(chunks):
    print(f"\n--- Chunk {i} ---")
    print(chunk[:300])
```

6.7 Replace the Make Iterator with a Python loop

```
for i, chunk in enumerate(chunks):
    print(f"Processing chunk {i + 1}/{len(chunks)}")
    vector = embed_text(chunk)
```

This is the same control flow as Make.com: the Iterator emits one chunk, then the Gemini and Pinecone modules run once for that chunk. Python expresses the same behavior with a for loop.

6.8 Upsert each vector into Pinecone

```
index.upsert(
    namespace=NAMESPACE,
    vectors=[
        {
            "id": f"{file_path.name}-chunk-{i}",
            "values": vector,
            "metadata": {
                "text": chunk,
                "source": file_path.name,
                "chunk_number": i,
            },
        },
    ],
)
```

Each Pinecone record now contains:

- id - a unique identifier such as intelligent_agents_overview.txt-chunk-0;
- values - the 1024-dimensional Gemini embedding;
- metadata.text - the original chunk text;
- metadata.source - the original file name;
- metadata.chunk_number - the chunk position.

6.9 Run Part 1

```
python ingest.py
```

Expected pattern of output:

```

Loaded: intelligent_agents_overview.txt
Characters: ...
Created ... chunks
Processing chunk 1/...
Processing chunk 2/...
...
Ingestion complete.

```

6.10 Check Pinecone

1. Open your Pinecone console.
2. Open the index named in PINECONE_INDEX.
3. Select or inspect the namespace week3-code-lab.
4. Open one record and confirm that the vector exists.
5. Confirm that metadata contains text, source, and chunk_number.

Do not continue blindly: If Part 1 finishes but you cannot see records or metadata in Pinecone, fix ingestion first. Part 2 depends entirely on those records.

7. Part 2 - Build the Query Pipeline

Question -> Embed -> Pinecone query -> Top 3 chunks -> Aggregate -> Gemini -> Answer

7.1 Use the same configuration

```

NAMESPACE = "week3-code-lab"
EMBEDDING_MODEL = "gemini-embedding-2"
EMBEDDING_DIMENSION = 1024
GENERATION_MODEL = "gemini-3.6-flash"
TOP_K = 3

```

Critical consistency: The query pipeline must use the same Pinecone index, namespace, embedding model, and embedding dimension used during ingestion. Otherwise retrieval may fail or produce meaningless comparisons.

7.2 Get a question

```

question = input("Ask a question: ").strip()
if not question:
    raise ValueError("Question cannot be empty")

```

Make mapping: Tools -> Set variable.

7.3 Embed the question

```

query_vector = embed_text(question)
print(f"Query embedding dimensions: {len(query_vector)}")

```

Question text -> Gemini Embedding 2 -> 1024-dimensional query vector

7.4 Search Pinecone

```
results = index.query(
    namespace=NAMESPACE,
    vector=query_vector,
    top_k=TOP_K,
    include_metadata=True,
    include_values=False,
)
```

What top_k=3 means: return the three stored vectors that are most similar to the question vector. This mirrors the Limit = 3 setting in the Make query pipeline.

7.5 Inspect retrieval before generating an answer

```
print("\nRetrieved chunks:")
context_parts = []

for rank, match in enumerate(results.matches, start=1):
    text = match.metadata.get("text", "")
    source = match.metadata.get("source", "unknown")

    print("\n-----")
    print(f"Rank: {rank}")
    print(f"ID: {match.id}")
    print(f"Score: {match.score:.4f}")
    print(f"Source: {source}")
    print(f"Text: {text[:300]}")

    if text:
        context_parts.append(text)
```

This is the R in RAG: Do not hide the retrieved chunks while learning. A RAG system is not simply question -> LLM -> answer. You must be able to inspect which evidence was retrieved and whether it was actually relevant.

7.6 Aggregate the retrieved chunks

```
if not context_parts:
    print("\nNo context was retrieved from Pinecone.")
    raise SystemExit(1)

context = "\n\n---\n\n".join(context_parts)
```

Make mapping: Tools -> Text aggregator.

7.7 Generate a grounded response

```
interaction = gemini.interactions.create(
    model=GENERATION_MODEL,
    system_instruction={
        "Answer the user's question using only the supplied context. "
        "If the context does not contain enough information, say: "
        "'I don't have enough information in the provided documents.'"
    },
    input=f"CONTEXT:\n{context}\n\nQUESTION:\n{question}",
)

print("\n=====")
print("FINAL ANSWER")
print("=====")
print(interaction.output_text)
```

This is the augmentation and generation stage: the retrieved chunks are inserted into the model input as context, then Gemini generates an answer under the instruction that it must stay within that context.

8. Run and Test the Complete RAG System

8.1 Run ingestion first

```
python ingest.py
```

8.2 Then run the query pipeline

```
python query.py
```

8.3 Test 1 - Question covered by the document

Try a question related to intelligent agents, for example:

```
What is a learning agent?
```

Check both outputs:

- Were the retrieved chunks relevant to the question?
- Did the final answer use information visible in those chunks?

8.4 Test 2 - Question not covered by the document

```
What's the weather in Melbourne?
```

Expected behavior: the model should say that the supplied documents do not contain enough information instead of inventing an answer.

Important limitation: A prompt that says "use only the context" reduces hallucination but does not mathematically guarantee it. Always inspect retrieval and evaluate whether the answer is actually grounded.

9. Required Experiments

Experiment A - Change top-k

Run the same question with:

```
TOP_K = 1
TOP_K = 3
TOP_K = 5
```

Compare:

- Which chunks were retrieved?
- Did additional chunks improve the answer or add irrelevant context?
- Was the answer more focused with a smaller top-k?

Experiment B - Change chunk size

Try:

```
chunks = chunk_text(text, chunk_size=100, overlap=20)
# then compare with
chunks = chunk_text(text, chunk_size=400, overlap=20)
```

Use a fresh namespace for each chunking experiment: If the number of chunks changes, old record IDs can remain in the namespace. The simplest safe method for this lab is to change NAMESPACE to values such as week3-code-100 and week3-code-400 before re-ingesting.

Compare:

- How many chunks were created?
- Did smaller chunks retrieve more precise passages?
- Did larger chunks provide more context but include more irrelevant text?

Experiment C - Explain the abstraction change

For each operation below, write the corresponding Make.com module and Python statement:

Operation	Make.com	Python
Load document		
Chunk text		
Iterate chunks		
Embed text		
Store vector		
Embed question		
Retrieve top-k		
Aggregate context		
Generate answer		

10. Troubleshooting

Problem / error	Likely cause	What to check
ModuleNotFoundError	Package not installed in the active environment	Activate .venv, then run <code>pip install -U google-genai pinecone python-dotenv</code>
Missing API key ValueError	.env missing or variable name incorrect	Check spelling and that .env is in the project root
FileNotFoundError	Text file is not in data/	Confirm data/intelligent_agents_overview.txt exists
Vector dimension mismatch	Pinecone index dimension does not match embedding length	Use 1024 in the index and both Python pipelines
No records / no results	Wrong index or namespace, ingestion did not complete	Check PINECONE_INDEX and NAMESPACE, then inspect Pinecone
Results appear immediately inconsistent after upsert	Pinecone writes may be eventually consistent	Wait briefly and retry the query
Model not found	Gemini model name has changed or is unavailable to your account	Check the current official Gemini model documentation
Old chunks appear after changing chunk size	Upsert overwrites matching IDs but does not remove unrelated old IDs	Use a fresh namespace or delete old records before re-ingesting

11. What You Should Be Able to Explain

- Why do we chunk a document instead of embedding the entire document as one vector?
- What does the embedding model produce?
- Why must the document chunks and the question use the same embedding model and dimension?
- What does Pinecone store for each record?
- What does cosine similarity help us find?
- What does top-k control?
- Why do we store the original chunk text as metadata if Pinecone already stores the vector?
- Where does retrieval end and generation begin?
- What is the purpose of the system instruction that says to answer only from context?
- What part of the Python implementation corresponds to the Make Iterator?

Minimal mental model: Ingestion creates searchable knowledge. Query retrieves the most relevant knowledge. Generation turns that retrieved knowledge into a natural-language answer.

12. Student Deliverable

Submit the following evidence:

- Screenshot of a Pinecone record from the Python namespace showing stored metadata.
- Screenshot of a query.py run showing the retrieved chunks and final answer.
- Result of one out-of-scope question and a short explanation of whether the system hallucinated.
- One parameter modification (top-k or chunk size) with a short before/after comparison.

- A short mapping of Make.com modules to the equivalent Python operations.

13. Appendix A - Complete ingest.py

```
import os
from pathlib import Path

from dotenv import load_dotenv
from google import genai
from google.genai import types
from pinecone import Pinecone

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
INDEX_NAME = os.getenv("PINECONE_INDEX")

NAMESPACE = "week3-code-lab"
EMBEDDING_MODEL = "gemini-embedding-2"
EMBEDDING_DIMENSION = 1024

if not all([GEMINI_API_KEY, PINECONE_API_KEY, INDEX_NAME]):
    raise ValueError("Missing GEMINI_API_KEY, PINECONE_API_KEY, or PINECONE_INDEX in .env")

gemini = genai.Client(api_key=GEMINI_API_KEY)
pc = Pinecone(api_key=PINECONE_API_KEY)
index = pc.Index(INDEX_NAME)

def chunk_text(text: str, chunk_size: int = 200, overlap: int = 20) -> list[str]:
    """Split text into overlapping word-based chunks."""
    if chunk_size <= 0:
        raise ValueError("chunk_size must be greater than 0")
    if overlap < 0 or overlap >= chunk_size:
        raise ValueError("overlap must be >= 0 and smaller than chunk_size")

    words = text.split()
    chunks = []
    step = chunk_size - overlap

    for start in range(0, len(words), step):
        end = start + chunk_size
        chunk_words = words[start:end]

        if not chunk_words:
            break

        chunks.append(" ".join(chunk_words))

    if end >= len(words):
        break

    return chunks

def embed_text(text: str) -> list[float]:
    result = gemini.models.embed_content(
        model=EMBEDDING_MODEL,
        contents=text,
        config=types.EmbedContentConfig(
            output_dimensionality=EMBEDDING_DIMENSION
        ),
    )
    return result.embeddings[0].values

file_path = Path("data/intelligent_agents_overview.txt")

if not file_path.exists():
    raise FileNotFoundError(f"Cannot find {file_path}. Put the lab text file inside the data folder.")
```

```

text = file_path.read_text(encoding="utf-8")
print(f"Loaded: {file_path.name}")
print(f"Characters: {len(text)}")

chunks = chunk_text(text, chunk_size=200, overlap=20)
print(f"Created {len(chunks)} chunks")

for i, chunk in enumerate(chunks):
    print(f"Processing chunk {i + 1}/{len(chunks)}")
    vector = embed_text(chunk)

    index.upsert(
        namespace=NAMESPACE,
        vectors=[
            {
                "id": f"{file_path.name}-chunk-{i}",
                "values": vector,
                "metadata": {
                    "text": chunk,
                    "source": file_path.name,
                    "chunk_number": i,
                },
            },
        ],
    )

print("Ingestion complete.")

```

14. Appendix B - Complete query.py

```

import os

from dotenv import load_dotenv
from google import genai
from google.genai import types
from pinecone import Pinecone

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
INDEX_NAME = os.getenv("PINECONE_INDEX")

NAMESPACE = "week3-code-lab"
EMBEDDING_MODEL = "gemini-embedding-2"
EMBEDDING_DIMENSION = 1024
GENERATION_MODEL = "gemini-3.6-flash"
TOP_K = 3

if not all([GEMINI_API_KEY, PINECONE_API_KEY, INDEX_NAME]):
    raise ValueError("Missing GEMINI_API_KEY, PINECONE_API_KEY, or PINECONE_INDEX in .env")

gemini = genai.Client(api_key=GEMINI_API_KEY)
pc = Pinecone(api_key=PINECONE_API_KEY)
index = pc.Index(INDEX_NAME)

def embed_text(text: str) -> list[float]:
    result = gemini.models.embed_content(
        model=EMBEDDING_MODEL,
        contents=text,
        config=types.EmbedContentConfig(
            output_dimensionality=EMBEDDING_DIMENSION
        ),
    )
    return result.embeddings[0].values

question = input("Ask a question: ").strip()
if not question:

```

```

raise ValueError("Question cannot be empty")

query_vector = embed_text(question)
print(f"Query embedding dimensions: {len(query_vector)}")

results = index.query(
    namespace=NAMESPACE,
    vector=query_vector,
    top_k=TOP_K,
    include_metadata=True,
    include_values=False,
)

print("\nRetrieved chunks:")
context_parts = []

for rank, match in enumerate(results.matches, start=1):
    text = match.metadata.get("text", "")
    source = match.metadata.get("source", "unknown")

    print("\n-----")
    print(f"Rank: {rank}")
    print(f"ID: {match.id}")
    print(f"Score: {match.score:.4f}")
    print(f"Source: {source}")
    print(f"Text: {text[:300]}")

    if text:
        context_parts.append(text)

if not context_parts:
    print("\nNo context was retrieved from Pinecone.")
    raise SystemExit(1)

context = "\n\n---\n\n".join(context_parts)

interaction = gemini.interactions.create(
    model=GENERATION_MODEL,
    system_instruction=(
        "Answer the user's question using only the supplied context. "
        "If the context does not contain enough information, say: "
        "'I don't have enough information in the provided documents.'"
    ),
    input=f"CONTEXT:\n{context}\n\nQUESTION:\n{question}",
)

print("\n=====")
print("FINAL ANSWER")
print("=====")
print(interaction.output_text)

```

15. Appendix C - Pipeline Summary

MAKE.COM PART 1		PYTHON PART 1
Download a File	->	Path.read_text()
Chunk text	->	chunk_text()
Iterator	->	for loop
Generate embeddings	->	embed_text(chunk)
Upsert vector	->	index.upsert()

MAKE.COM PART 2	PYTHON PART 2
Set variable	-> input()
Generate embedding	-> embed_text(question)
Pinecone search	-> index.query()
Text aggregator	-> join()
Generate response	-> gemini.interactions.create()
Get variable	-> print()

16. References and Version Notes

The code patterns and API notes in this guide were checked against the following:

- [Google AI for Developers - Gemini Embeddings](#)
- [Google AI for Developers - Text Generation / Interactions API](#)
- [Pinecone - Create an Index](#)
- [Pinecone - Data Modeling](#)
- [Pinecone - Upsert Data](#)

Course-source note: this lab is designed to mirror the supplied Week 3 Make.com RAG ingestion and query scenarios, including 200/20 chunk settings, 1024-dimensional embeddings, top-k=3, and Pinecone metadata storage.

Version note: Cloud SDKs and model names change over time. If a model name or method is unavailable in a future semester, update only the API-specific call while preserving the RAG pipeline logic: chunk -> embed -> store -> retrieve -> augment -> generate.